ReactJS Fundamentals - Lecture Notes

1. What is ReactJS & Why do we need it?

ReactJS is a declarative, efficient, and flexible JavaScript library for building user interfaces. It allows developers to build complex, interactive UIs from small, isolated pieces of code called 'components'.

Why do we need it?
Before React, building dynamic web applications required manual DOM manipulation, which was slow, buggy, and hard to maintain. React solves this by providing a Component-Based Architecture and a Virtual DOM, making UI updates highly efficient and code highly reusable.

2. Advantages and Disadvantages of ReactJS

Advantages:

- Component Reusability: Write once, use everywhere.
- Performance: Fast rendering using the Virtual DOM.
- Rich Ecosystem: Massive community, tools, and libraries.

Disadvantages:

- Fast-Paced Changes: Constant updates require continuous learning.
- SEO Limitations: Traditional React (CSR) struggles with SEO out of the box.
- JSX Learning Curve: Mixing HTML with JavaScript can be daunting initially.

3. Vite & Project Setup

Vite is a modern frontend build tool that is significantly faster than Create React App (CRA). It offers instant server start and lightning-fast Hot Module Replacement (HMR).

Steps to setup a React project with Vite:

1. npm create vite@latest (Initializes the project)

2. cd <project-name>

3. npm i (Installs all required dependencies from package.json)

4. npm run dev (Starts the local development server)

4. Dependencies vs Dev Dependencies

Dependencies: Libraries required for your application to run in production (e.g., react, react-dom).

Dev Dependencies: Tools needed only during development and testing, not in the final production build (e.g., vite, eslint).

5. SPA vs MPA (Single vs Multi-Page Applications)

- SPA (Single Page Application): The server sends a single HTML file. Routing and UI updates happen purely via JavaScript without reloading the page.
- MPA (Multi-Page Application): Every time you click a link, the browser fetches a completely new HTML page from the server.

6. CSR vs SSR (Client-Side vs Server-Side Rendering)

Feature	CSR	SSR
Rendering	In the user's browser using JS	On the web server
Initial Load	Slower (must download JS first)	Faster (pre-rendered HTML)
SEO	Poor (crawlers see blank page initially)	Excellent (fully formed HTML)
Subsequent Loads	Very fast	Slower (requires new server request)

7. Babel

Babel is a JavaScript compiler. Browsers don't understand JSX natively. Babel transforms JSX and modern ES6+ code into plain, backward-compatible JavaScript that any browser can execute.

8. Virtual DOM & How It Works

The Virtual DOM is a lightweight in-memory representation of the Real DOM. Updating the Real DOM directly is slow. React uses a highly optimized tree-diffing algorithm to handle updates efficiently. This tree-diffing logic operates much like standard algorithmic checks on core data structures to minimize operational cost.

- The Reconciliation Process:
- 1. Render: React creates a new Virtual DOM tree.
- 2. Diffing: React compares the new tree with the previous Virtual DOM tree.
- 3. Commit: React calculates the minimum changes required and updates ONLY those specific nodes in the Real DOM.

9. Class-Based vs Functional Components

- Class Components: The older way. Written as ES6 classes requiring a render() method. They involve more boilerplate and require understanding the 'this' context.
- Functional Components: The modern way. Just plain JavaScript functions that return JSX. Thanks to Hooks, they can now handle state and lifecycle events, entirely replacing the need for classes.

10. Props in React

Props (Properties) are used to pass data from a parent component to a child component. Just like passing arguments into a standard C++ function, props supply the component with the inputs it needs to render. They are read-only (immutable), so child components cannot alter them.

11. React Hooks

Hooks are special functions introduced in React 16.8 that allow you to 'hook into' React state and lifecycle features from functional components. Before hooks, managing state or side effects required bulky Class Components. Hooks make code simpler, cleaner, and more reusable, avoiding confusion with the 'this' keyword.

A. useState Hook

Used to create and manage local variables (state) inside a component. When state updates, React automatically re-renders the component to reflect the new data.

```
import React, { useState } from 'react';

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <div>
      <p>Current Count: {count}</p>
      <button onClick={() => setCount(count + 1)}>Increase</button>
      <button onClick={() => setCount(count - 1)}>Decrease</button>
    </div>
  );
}
export default Counter;
```

B. useRef Hook

Used to access direct DOM elements or keep mutable variables without triggering a re-render. Unlike useState, updating a ref does NOT cause the component to update.

```
import React, { useRef } from 'react';

function FocusInput() {
  const inputRef = useRef(null); // Reference to the input element

  const handleFocus = () => {
    inputRef.current.focus(); // Focuses the input element
    alert("Input value: " + inputRef.current.value); // Reads value
directly
  };

  return (
    <div>
      <input ref={inputRef} type="text" placeholder="Type here..." />
      <button onClick={handleFocus}>Focus & Read Input</button>
    </div>
  );
}
export default FocusInput;
```

C. useEffect Hook

Used to perform side effects in functional components. Examples include fetching data, timers, or subscribing to events.

Lifecycle Phases in useEffect:

1. Mounting: Component is created and inserted into the DOM. (Runs the effect).
2. Updation: Component's state/props change. (Controlled via the dependency array).
3. Unmounting: Component is removed from the DOM. (Managed by returning a cleanup function).

The Dependency Array:

- useEffect(() => {...}) -> Runs on EVERY render.
- useEffect(() => {...}, []) -> Runs ONLY ONCE when component mounts.
- useEffect(() => {...}, [data]) -> Runs when component mounts AND whenever 'data' changes.

```
import React, { useState, useEffect } from 'react';

function DataFetcher() {
  const [data, setData] = useState([]);

  useEffect(() => {
    // 1. MOUNTING / UPDATION: Fetching data from a fake API
    fetch('https://jsonplaceholder.typicode.com/users')
      .then(response => response.json())
      .then(data => setData(data));
```

```
    // 3. UNMOUNTING: Cleanup function
    return () => {
      console.log("Component is unmounting or effect is cleaning up");
    };
  }, []); // 2. Empty array: fetch only runs once on initial load

  return (
    <ul>
      {data.map(user => (
        <li key={user.id}>{user.name}</li>
      ))}
    </ul>
  );
}
export default DataFetcher;
```

12. Full Project: Todo Application (Props & LocalStorage)

This example breaks a basic Todo application into three functional components. It demonstrates passing props, calling parent functions from a child component, and using the browser's localStorage to save tasks.

1. App.jsx (Main Component)

Holds the state and manages adding/deleting. It uses useEffect to sync with localStorage.

```
import React, { useState, useEffect } from 'react';
import TodoForm from './TodoForm';
import TodoItem from './TodoItem';

function App() {
  const [todos, setTodos] = useState([]);

  // Load from LocalStorage on mount
  useEffect(() => {
    const savedTodos = JSON.parse(localStorage.getItem('todos')) || [];
    setTodos(savedTodos);
  }, []);

  // Save to LocalStorage whenever 'todos' array changes
  useEffect(() => {
    localStorage.setItem('todos', JSON.stringify(todos));
  }, [todos]);

  const addTodo = (title, description) => {
    const newTodo = { id: Date.now(), title, description };
    setTodos([...todos, newTodo]); // Update state
  };

  const deleteTodo = (id) => {
    setTodos(todos.filter(todo => todo.id !== id));
```

```
  };

  return (
    <div style={{ padding: '20px', maxWidth: '500px' }}>
      <h2>My Todo List</h2>
      {/* Passing function as a prop to child */}
      <TodoForm addTodo={addTodo} />

      <div>
        {todos.map(todo => (
           /* Passing data and functions as props */
          <TodoItem key={todo.id} todo={todo} deleteTodo={deleteTodo} />
        ))}
      </div>
    </div>
  );
}
export default App;
```

2. TodoForm.jsx (Child Component)

Takes inputs and calls the parent's `addTodo` function via props.

```
import React, { useState } from 'react';

function TodoForm({ addTodo }) {
  const [title, setTitle] = useState('');
  const [desc, setDesc] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    if (!title || !desc) return; // Validation

    addTodo(title, desc); // Call parent function

    // Clear inputs
    setTitle('');
    setDesc('');
  };

  return (
    <form onSubmit={handleSubmit} style={{ marginBottom: '20px' }}>
      <input
        type="text"
        placeholder="Task Name"
        value={title}
        onChange={(e) => setTitle(e.target.value)}
        required
      />
      <input
```

```
          type="text"
          placeholder="Description"
          value={desc}
          onChange={(e) => setDesc(e.target.value)}
          required
        />
        <button type="submit">Add Task</button>
      </form>
    );
}
export default TodoForm;
```

3. TodoItem.jsx (Child Component)
Receives a specific todo object and the delete function to render the UI.

```
import React from 'react';

function TodoItem({ todo, deleteTodo }) {
  return (
    <div style={{ border: '1px solid #ccc', margin: '10px 0', padding:
'10px' }}>
      <h3 style={{ margin: '0 0 5px 0' }}>{todo.title}</h3>
      <p style={{ margin: '0 0 10px 0' }}>{todo.description}</p>

      {/* Trigger parent function using the current todo's ID */}
      <button onClick={() => deleteTodo(todo.id)}>Delete</button>
    </div>
  );
}
export default TodoItem;
```

13. Prop Drilling

Prop Drilling is the process of passing data from a higher-level component down to a lower-level component through multiple intermediate components that do not actually need the data themselves. It makes the code verbose and harder to maintain.

```
import React, { useState } from 'react';

// App has the data, but Grandchild needs it.
function App() {
  const [userName, setUserName] = useState("John Doe");
  return <Parent name={userName} />;
}

// Parent doesn't need 'name', but must pass it down.
function Parent({ name }) {
  return <Child name={name} />;
}
```

```
// Child doesn't need 'name', but must pass it down.
function Child({ name }) {
  return <Grandchild name={name} />;
}

// Grandchild finally uses it.
function Grandchild({ name }) {
  return <h3>Hello, {name}!</h3>;
}
export default App;
```

14. Context API & useContext Hook

The Context API solves the Prop Drilling problem. It allows you to 'teleport' data directly to the component that needs it, bypassing the intermediate components.

```
import React, { useState, createContext, useContext } from 'react';

// 1. Create the Context (Export it so others can use it)
const UserContext = createContext();

function App() {
  const [userName, setUserName] = useState("John Doe");

  return (
    // 2. Provide the context value to the entire tree
    <UserContext.Provider value={userName}>
      <Parent />
    </UserContext.Provider>
  );
}

// Notice: Parent and Child no longer take or pass 'name' props!
function Parent() { return <Child />; }
function Child() { return <Grandchild />; }

function Grandchild() {
  // 3. Consume the context using the useContext hook
  const name = useContext(UserContext);
  return <h3>Hello directly to {name}!</h3>;
}
export default App;
```

15. Redux Toolkit (RTK)

What is it?: Redux Toolkit is the official, recommended, and modern way to write Redux logic. Redux is a global state management library.
Why do we need it?: For large applications, Context API can cause unnecessary re-renders.

Redux provides a highly optimized, centralized global store where any component can access or update data.

Workflow Diagram (Counter Example):
UI Component (Clicks '+') --> Dispatch(Action: increment) --> Reducer updates State --> Store saves new State --> UI Re-renders

Core Terms Definitions:
• **State:** The actual data of your application (e.g., count: 0).
• **Store:** The centralized database for your frontend. It holds all the state for the entire app.
• **Slice:** A collection of Redux reducer logic and actions for a single feature (e.g., a 'counterSlice'). It groups the feature's state and logic together.
• **Action:** An event that describes something happening. It can be just a trigger (Event) like 'increment', or it can carry extra data (Event + Add. Info) like 'incrementByAmount(5)'. This extra info is called the payload.
• **Reducer:** The logic/function that takes the current State and the Action, and returns the newly updated State. It is the ONLY thing allowed to change the state.

Full Redux Toolkit Basic Code Example:

```
// 1. counterSlice.js (Defining the Slice, State, and Reducers)
import { createSlice } from '@reduxjs/toolkit';

const counterSlice = createSlice({
  name: 'counter',       // Name of the slice
  initialState: {        // The initial STATE
    value: 0
  },
  reducers: {            // The REDUCERS (logic to update state)
    increment: (state) => {
      state.value += 1;  // RTK allows "mutating" state safely
    },
    decrement: (state) => {
      state.value -= 1;
    },
    // Action with Payload (Event + Add. Info)
    incrementByAmount: (state, action) => {
      state.value += action.payload;
    }
  }
});

// Export ACTIONS so components can dispatch them
export const { increment, decrement, incrementByAmount } =
counterSlice.actions;
```

```
// Export REDUCER so the store can use it
export default counterSlice.reducer;

// 2. store.js (Creating the STORE)
import { configureStore } from '@reduxjs/toolkit';
import counterReducer from './counterSlice';

export const store = configureStore({
  reducer: {
    counter: counterReducer, // Add the slice to the store
  },
});

// 3. main.jsx (Connecting Store to the App)
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';
import { Provider } from 'react-redux';
import { store } from './store';

ReactDOM.createRoot(document.getElementById('root')).render(
  // Provider makes the store available to any component inside App
  <Provider store={store}>
    <App />
  </Provider>
);

// 4. Counter.jsx (Using Redux in a Component)
import React from 'react';
import { useSelector, useDispatch } from 'react-redux';
import { increment, decrement, incrementByAmount } from
'./counterSlice';

function Counter() {
  // Read STATE from store using useSelector
  const count = useSelector((state) => state.counter.value);

  // useDispatch allows us to send actions to the reducers
  const dispatch = useDispatch();

  return (
    <div>
      <h2>Global Count: {count}</h2>
      {/* Dispatching ACTIONS */}
      <button onClick={() => dispatch(increment())}>+</button>
      <button onClick={() => dispatch(decrement())}>-</button>
      <button onClick={() => dispatch(incrementByAmount(5))}>Add
5</button>
    </div>
  );
}
export default Counter;
```

16. Custom Hooks

What are they?: Custom Hooks are standard JavaScript functions whose names start with 'use' (e.g., useToggle, useFetch) and that call other React Hooks. They allow you to extract and share stateful logic between different components.

Where to use them?: Whenever you find yourself writing the same hook logic (like fetching data, managing a modal's open/close state, or tracking window size) across multiple components. Custom hooks keep your components clean and DRY (Don't Repeat Yourself).

```
import React, { useState } from 'react';

// 1. Creating the Custom Hook (useToggle)
function useToggle(initialValue = false) {
  const [value, setValue] = useState(initialValue);

  const toggle = () => {
    setValue((prevValue) => !prevValue);
  };

  // Return the state and the toggle function
  return [value, toggle];
}

// 2. Using the Custom Hook in a Component
function ToggleComponent() {
  // Using our custom hook for a Hide/Show feature
  const [isVisible, toggleVisibility] = useToggle(false);

  return (
    <div>
      <button onClick={toggleVisibility}>
        {isVisible ? 'Hide' : 'Show'} Secret Message
      </button>

      {isVisible && <p>Hello! This is the secret message.</p>}
    </div>
  );
}
export default ToggleComponent;
```

17. React Routing (react-router-dom)

React builds Single Page Applications (SPAs). To navigate between different views (like Home, About, Contact) without reloading the page, we use an external library called **react-router-dom**.

Core Components:
• **BrowserRouter:** The parent wrapper that enables routing for the entire app. It uses the HTML5 history API to keep UI in sync with the URL.
• **Routes:** Acts as a container/switch for all your individual routes. It looks through its children <Route>s and renders the one that matches the current URL.
• **Route:** Maps a specific URL path to a specific React Component.
• **Link:** The React Router equivalent of an HTML <a> tag. It changes the URL without causing a full page refresh.

```
import React from 'react';
// Import necessary components from react-router-dom
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';

// Dummy Components for different pages
const Home = () => <h2>Home Page</h2>;
const About = () => <h2>About Page</h2>;
const NotFound = () => <h2>404 - Page Not Found</h2>;

function App() {
  return (
    // 1. Wrap the app in BrowserRouter
    <BrowserRouter>
      <nav>
        {/* 2. Use Link for navigation instead of <a href="..."> */}
        <Link to="/">Home</Link> | <Link to="/about">About</Link>
      </nav>

      {/* 3. Define the Routes container */}
      <Routes>
        {/* 4. Map specific paths to components */}
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
        {/* Catch-all route for 404s */}
        <Route path="*" element={<NotFound />} />
      </Routes>
    </BrowserRouter>
  );
}
export default App;
```

18. API Callings in React (CRUD Operations)

Interacting with backend servers is typically done using the built-in Fetch API or external libraries like Axios. Here is how you map standard HTTP methods to CRUD (Create, Read, Update, Delete) operations.

Definitions:
• **GET:** Retrieve data from the server.
• **POST:** Send new data to the server to be created.

- **PUT:** Update an entire existing resource.
- **PATCH:** Update only specific fields of an existing resource.
- **DELETE:** Remove a resource from the server.

```
// Example of basic API calls inside a React component using async/await

const API_URL = 'https://jsonplaceholder.typicode.com/posts';

// 1. GET (Read Data)
const fetchData = async () => {
  try {
    const response = await fetch(API_URL);
    const data = await response.json();
    console.log("GET Result:", data);
  } catch (error) { console.error("Error fetching data:", error); }
};

// 2. POST (Create Data)
const createData = async () => {
  try {
    const response = await fetch(API_URL, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'New Post', body: 'This is my post',
userId: 1 })
    });
    const data = await response.json();
    console.log("POST Result:", data);
  } catch (error) { console.error("Error creating data:", error); }
};

// 3. PUT (Update Full Data)
const updateFullData = async (id) => {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ id, title: 'Updated Title', body: 'Updated
body', userId: 1 })
    });
    const data = await response.json();
    console.log("PUT Result:", data);
  } catch (error) { console.error("Error updating data:", error); }
};

// 4. PATCH (Update Partial Data)
const updatePartialData = async (id) => {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'PATCH',
```

```
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ title: 'Only Title Changed' }) // Only
sending title
    });
    const data = await response.json();
    console.log("PATCH Result:", data);
  } catch (error) { console.error("Error patching data:", error); }
};

// 5. DELETE (Remove Data)
const deleteData = async (id) => {
  try {
    const response = await fetch(`${API_URL}/${id}`, {
      method: 'DELETE'
    });
    console.log("DELETE Result Status:", response.status); // Usually
200 or 204
  } catch (error) { console.error("Error deleting data:", error); }
};
```